



UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE MANTENIMIENTO DE SOFTWARE GUION DE PRESENTACIÓN DE PROPUESTA

PROYECTO: YOLIZTLI

EQUIPO DE MANTENIMIENTO:

CHAVARRÍA ÁNGELES BRANDON
ISLAS LOZADA EMMANUEL
MORÁN GÓMEZ KENNETH
SÁNCHEZ ROMERO JOSÉ ENRIQUE

CATEDRÁTICO:

ALICIA ORTIZ MONTES

PACHUCA DE SOTO, HIDALGO 13 DE JUNIO DE 2026



ÍNDICE

1. Introducción y Estructura del Video	2
1.1. Instrucciones Técnicas para la Grabación	2
2. Guion Técnico (Script del Video)	3
2.1. Parte 1: Introducción, Contexto y Marco Metodológico (0:00 - 1:15) — Emmanuel Islas	3
2.2. Parte 2: Diagnóstico mediante Análisis Estático con SonarQube (1:15 - 2:30) — Brandon Chavarría	4
2.3. Parte 3: Gestión de PR (Problem Reports) y MR (Modification Requests) (2:30 - 3:45) — Kenneth Morán	5
2.4. Parte 4: Estimación de Esfuerzo, Gestión SCM y Conclusión (3:45 - 5:00) — José Enrique Sánchez	6

I INTRODUCCIÓN Y ESTRUCTURA DEL VIDEO

El propósito de este documento es proporcionar el libreto y guion técnico detallado para la grabación del video de presentación de la propuesta de mantenimiento del sistema educativo **Yoliztli**.

Participante	Rol en la Presentación	Módulo Temático
Emmanuel Islas	Coordinador del Proyecto	Introducción, contexto y metodología.
Brandon Chavarría	Ingeniero de Pruebas	Análisis estático de código y SonarQube.
Kenneth Morán	Ingeniero de Software	Reportes de Problemas y Modificaciones.
José Enrique Sánchez	Gestor de Configuración	Estimación de esfuerzo, SCM y Cierre.

Tabla 1. Roles y distribución del equipo en el video

I.1 INSTRUCCIONES TÉCNICAS PARA LA GRABACIÓN

- **Duración total objetivo:** 4 a 6 minutos.
- **Léxico:** Formal y académico, empleando terminología estándar de ingeniería de software (SWEBOK v4.0 e ISO/IEC/IEEE 14764).
- **Apoyo visual:** Se grabará la pantalla mostrando las herramientas de diagnóstico (SonarQube) y las secciones clave de los PDFs generados.



2 GUION TÉCNICO (SCRIPT DEL VIDEO)

2.1 PARTE I: INTRODUCCIÓN, CONTEXTO Y MARCO METODOLÓGICO

(0:00 - 1:15) — EMMANUEL ISLAS

[Visual de apoyo]: *Emmanuel a cámara y luego compartiendo pantalla con la portada del Plan de Mantenimiento de Software.*

Emmanuel: Hola a todos. A nombre del equipo de mantenimiento del proyecto **Yoliztli**, presentamos nuestra propuesta formal para la evolución y estabilización del software. Yoliztli es una aplicación web educativa diseñada para el aprendizaje infantil de lenguas originarias, estructurada sobre una arquitectura monolítica híbrida empleando **Laravel 10** en el backend, **React.js** en el frontend y la biblioteca de comunicación reactiva **Inertia.js**.

Nuestra estrategia de mantenimiento está rigurosamente alineada con la guía del cuerpo de conocimiento de ingeniería de software **SWEBOK v4.0** y el estándar internacional **ISO/IEC/IEEE 14764**. Hemos clasificado las intervenciones requeridas en cuatro categorías principales: correctivo, perfectivo, adaptativo y aditivo. El punto de partida de este proceso ha sido la ejecución de pruebas estáticas de código para evaluar la calidad del software entregado y mapear su deuda técnica. A continuación, mi compañero Brandon detallará este diagnóstico.



2.2 PARTE 2: DIAGNÓSTICO MEDIANTE ANÁLISIS ESTÁTICO CON SONARQUBE

(1:15 - 2:30) — BRANDON CHAVARRÍA

[Visual de apoyo]: *Captura de pantalla de la interfaz local de SonarQube mostrando las métricas del proyecto Yoliztli.*

Brandon: Gracias, Emmanuel. Para la auditoría de calidad de código, configuramos y levantamos una instancia local de **SonarQube Community Edition v25.2.0**. El análisis abarcó el código fuente escrito en PHP y JavaScript, revelando áreas críticas de intervención:

1. **Bugs de Fiabilidad:** Identificamos **67 bugs** que resultaron en una calificación de fiabilidad (Reliability) de **C**. Entre ellos destaca un bug de severidad **Alta** en el frontend y 6 de severidad **Media**, que representan vulnerabilidades en ejecución y potenciales excepciones no controladas.
2. ****Deuda Técnica e Mantenibilidad**:** Se detectaron **115 Code Smells**, equivalentes a una deuda técnica estimada de **6 horas y 54 minutos**, distribuidos en violaciones de consistencia (formato de archivos) y comentarios TODO pendientes de resolución.
3. ****Capa de Presentación Inoperativa**:** El hallazgo crítico original fue la ausencia total de las vistas React en el directorio `resources/js/Pages/`. Esta anomalía impedía la ejecución del sistema y obligó al equipo a implementar un mantenimiento correctivo de emergencia para reconstituir la interfaz web antes de proceder con las pruebas estáticas.

Le cedo la palabra a Kenneth, quien explicará cómo formalizamos estos hallazgos a través de reportes técnicos.



2.3 PARTE 3: GESTIÓN DE PR (PROBLEM REPORTS) Y MR (MODIFICATION REQUESTS)

(2:30 - 3:45) — KENNETH MORÁN

[Visual de apoyo]: *Kenneth mostrando los archivos `Problem_report.pdf` y `Modification_request.pdf`, resaltando las tablas de resumen de PR y MR.*

Kenneth: Gracias, Brandon. Siguiendo el flujo del proceso de mantenimiento establecido por el estándar ISO/IEC/IEEE 14764, documentamos cada hallazgo mediante un **Problem Report (PR)** y propusimos su solución con su respectivo **Modification Request (MR)**:

- **PR-001 y MR-001 (Capa de Presentación):** Catalogado como correctivo urgente, implicó la re-constitución de las vistas React y la corrección de errores de entrada en `public/index.php`, logrando pasar el estado del sistema de inoperativo a funcional.
- **PR-002 y MR-002 (Bugs de Fiabilidad):** Mantenimiento correctivo planificado para refactorizar los 67 bugs señalados en SonarQube, priorizando el bug de severidad Alta.
- **PR-003 y MR-003 (Estándares de Código):** Mantenimiento perfectivo enfocado en mitigar la deuda técnica. Proponemos automatizar el formato con herramientas de *linting* como **PHP-CS-Fixer** (bajo el estándar PSR-12) y **ESLint** para JavaScript.
- **PR-004 y MR-004 (Filtro por Perfil):** Derivado de la falla del caso de prueba de aceptación **CP007**. Consiste en un mantenimiento correctivo y aditivo en `DashboardController` para aplicar filtros SQL basados en la edad y la lengua configuradas por el alumno.
- **PR-005 y MR-005 (Registro de Progreso):** Asociado a la falla del caso **CP008**. Requiere la creación de un nuevo endpoint `POST /progress`, un controlador dedicado y una tabla en la base de datos para almacenar el progreso del estudiante de forma persistente.

A continuación, Enrique expondrá la estimación de esfuerzos y el control de calidad en la configuración.



2.4 PARTE 4: ESTIMACIÓN DE ESFUERZO, GESTIÓN SCM Y CONCLUSIÓN

(3:45 - 5:00) — JOSÉ ENRIQUE SÁNCHEZ

[Visual de apoyo]: *Enrique compartiendo pantalla con las fórmulas de estimación y el modelo de ramificación de Git.*

José Enrique: Gracias, Kenneth. Para garantizar la viabilidad y control de los cambios propuestos, realizamos estimaciones cuantitativas del esfuerzo requerido:

- **Estimación Funcional (COSMIC FFP):** Identificamos y medimos las funciones de movimiento de datos (entradas, salidas, lecturas y escrituras) de los nuevos módulos de progreso y filtros, totalizando el tamaño funcional en Puntos de Función COSMIC.
- **Estimación de Esfuerzo (COCOMO II):** Utilizando el modelo COCOMO para proyectos de tamaño pequeño a mediano, estimamos un esfuerzo total de desarrollo y mantenimiento correctivo/aditivo que nos permite proyectar un cronograma de ejecución controlado.

Asimismo, establecimos un esquema de **Gestión de la Configuración del Software (SCM)** utilizando Git. Para evitar la regresión de código, implementaremos un modelo de ramas donde las correcciones de fiabilidad se desarrollen en ramas `fix/`, las mejoras estéticas en `style/` o `refactor/`, y las nuevas características de filtrado y progreso en ramas `feature/`, requiriendo aprobación mediante **Pull Requests** verificados antes de integrarse a la rama principal.

Concluimos que la implementación integrada de este plan de mantenimiento no solo corregirá los fallos heredados del sistema de pruebas de aceptación, sino que elevará el estándar de calidad interna del software a niveles óptimos para su producción y posterior escalabilidad. Agradecemos su atención.